Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу «Операционные системы»

Группа: М8О-208БВ-25
Студент: Лунева Л.В.
Преподаватель: Бахарев В.Д.
Оценка: _____
Дата: 01.11.26

Москва, 2026

Постановка задачи

Вариант 13. Наложить K раз фильтр, использующий матрицу свёртки, на матрицу, состоящую из вещественных чисел. Размер окна задаётся пользователем.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы. Ограничение максимального количества потоков, работающих в один момент времени, должно задаваться ключом запуска программы.

Необходимо уметь продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы. В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков и объяснить полученные результаты.

Общий метод и алгоритм решения

Использованные системные вызовы и функции. Функции библиотеки POSIX Threads являются обёртками над системными вызовами ядра Linux: потоки создаются вызовом clone(), а ожидание на барьере реализовано через futex().

- `int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg);` – создаёт новый поток, который выполняет функцию start_routine с аргументом arg.
- `int pthread_join(pthread_t thread, void **retval);` – ожидает завершения потока.
- `int pthread_barrier_init(pthread_barrier_t *barrier, const pthread_barrierattr_t *attr, unsigned count);` – создаёт барьер на count потоков.
- `int pthread_barrier_wait(pthread_barrier_t *barrier);` – блокирует поток, пока до барьера не дойдут все count потоков.
- `int pthread_barrier_destroy(pthread_barrier_t *barrier);` – уничтожает барьер.

– `int clock_gettime(clockid_t clk_id, struct timespec *tp);` – возвращает время по монотонным часам CLOCK_MONOTONIC; используется для замеров.

– `pid_t getpid(void);` – возвращает идентификатор процесса, по которому командой ps -T -p <PID> можно посмотреть его потоки.

– `void *malloc(size_t size); void free(void *ptr);` – выделение и освобождение памяти под матрицы (внутри используют системные вызовы brk и mmap).

– `FILE *fopen(const char *path, const char *mode);` – открытие файла steps.txt, в который записываются шаги свёртки для маленьких матриц.

Свёртка

Каждый элемент новой матрицы равен взвешенной сумме элементов исходной матрицы в окне w×w с центром в этом элементе; веса берутся из матрицы свёртки (ядра):

$$dst[i][j] = \Sigma\,\Sigma\, src[i + di][j + dj] \cdot kernel[di + r][dj + r],$$

где di и dj пробегают значения от −r до r, а r = w / 2 — радиус окна. За краем матрицы берётся ближайший крайний элемент. В программе используется усредняющее ядро: все веса равны 1 / w^2. Размер окна w должен быть нечётным.

Фильтр применяется K раз: результат очередной итерации становится входом следующей. Поэтому используются два буфера: итерация читает из src, пишет в dst, после чего указатели меняются местами. Матрица хранится одним массивом по строкам, элемент (i, j) лежит по индексу i · cols + j.

Последовательная версия

Функция run_sequential K раз вызывает convolve_rows для всех строк матрицы и меняет буферы местами.

Параллельная версия

Строки матрицы делятся между T потоками поровну, первые rows mod T потоков получают на одну строку больше. Каждый поток читает общий src, но записывает только свои строки dst, поэтому гонок данных нет и мьютексы не нужны.

Потоки создаются один раз на все K итераций. После каждой итерации поток вызывает pthread_barrier_wait: следующую итерацию можно начинать

только тогда, когда все потоки дописали свои строки. Затем каждый поток меняет местами свои локальные копии указателей src и dst, поэтому одного барьера на итерацию достаточно.

Число потоков задаётся первым аргументом запуска: ./main <потоки> <строки> <столбцы> <окно> <K>. Программа запускает обе версии на одних и тех же данных, сравнивает результаты функцией memcmp и выводит время, ускорение и эффективность. Время измеряется функцией clock_gettime(CLOCK_MONOTONIC): функция clock() не подходит, так как суммирует процессорное время всех потоков. Время параллельной версии включает создание и завершение потоков.

Программа проверяет аргументы запуска и обрабатывает ошибки выделения памяти, создания барьера и потоков.

Код программы

main.c

```
#define _POSIX_C_SOURCE 200809L

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

int rows, cols;
int w;
int K;
double *kernel;
pthread_barrier_t barrier;

typedef struct {
    double *src, *dst;
    int start, end;
} ThreadArgs;

void die(const char *msg)
{
    fprintf(stderr, "Ошибка: %s\n", msg);
    exit(EXIT_FAILURE);
}

double now_ms(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
}

void print_matrix(FILE *f, const char *title, const double *m)
{
    fprintf(f, "%s:\n", title);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++)
            fprintf(f, "%8.2f ", m[i * cols + j]);
        fprintf(f, "\n");
```

```
    }
    fprintf(f, "\n");
}

int clamp(int v, int lo, int hi)
{
    if (v < lo) return lo;
    if (v > hi) return hi;
    return v;
}

void convolve_rows(const double *src, double *dst, int start, int end)
{
    int r = w / 2;

    for (int i = start; i < end; i++) {
        for (int j = 0; j < cols; j++) {
            double sum = 0.0;
            for (int di = -r; di <= r; di++) {
                for (int dj = -r; dj <= r; dj++) {
                    int y = clamp(i + di, 0, rows - 1);
                    int x = clamp(j + dj, 0, cols - 1);
                    sum += src[y * cols + x] * kernel[(di + r) * w + (dj + r)];
                }
            }
            dst[i * cols + j] = sum;
        }
    }
}

void run_sequential(double *src, double *dst)
{
    for (int k = 0; k < K; k++) {
        convolve_rows(src, dst, 0, rows);
        double *tmp = src; src = dst; dst = tmp;
    }
}

void save_steps(const char *filename, const double *input)
{
    FILE *f = fopen(filename, "w");
    if (f == NULL) {
        perror("Не удалось открыть файл для записи шагов");
        return;
    }

    size_t bytes = (size_t)rows * cols * sizeof(double);
    double *src = malloc(bytes), *dst = malloc(bytes);
    if (src == NULL || dst == NULL)
        die("не хватает памяти для записи шагов");
    memcpy(src, input, bytes);

    fprintf(f, "Матрица %dx%d, окно %dx%d, K = %d\n\n", rows, cols, w, w, K);
    print_matrix(f, "Исходная матрица", src);

    for (int k = 0; k < K; k++) {
        convolve_rows(src, dst, 0, rows);
        double *tmp = src; src = dst; dst = tmp;

        char title[64];
        snprintf(title, sizeof(title), "После итерации %d", k + 1);
        print_matrix(f, title, src);
    }

    fclose(f);
    free(src);
    free(dst);
    printf("Шаги свёртки записаны в файл %s\n", filename);
}

void *worker(void *arg)
```

```
{
    ThreadArgs *t = arg;
    double *src = t->src, *dst = t->dst;

    for (int k = 0; k < K; k++) {
        convolve_rows(src, dst, t->start, t->end);
        pthread_barrier_wait(&barrier);
        double *tmp = src; src = dst; dst = tmp;
    }
    return NULL;
}

void run_parallel(double *src, double *dst, int threads)
{
    pthread_t *ids = malloc(threads * sizeof(pthread_t));
    ThreadArgs *args = malloc(threads * sizeof(ThreadArgs));
    if (ids == NULL || args == NULL)
        die("не хватает памяти под потоки");

    if (pthread_barrier_init(&barrier, NULL, threads) != 0)
        die("не удалось создать барьер");

    int per_thread = rows / threads;
    int extra = rows % threads;
    int start = 0;

    for (int i = 0; i < threads; i++) {
        int count = per_thread + (i < extra ? 1 : 0);
        args[i].src = src;
        args[i].dst = dst;
        args[i].start = start;
        args[i].end = start + count;
        start += count;

        int rc = pthread_create(&ids[i], NULL, worker, &args[i]);
        if (rc != 0) {
            fprintf(stderr, "Ошибка pthread_create: %s\n", strerror(rc));
            exit(EXIT_FAILURE);
        }
    }

    for (int i = 0; i < threads; i++)
        pthread_join(ids[i], NULL);

    pthread_barrier_destroy(&barrier);
    free(ids);
    free(args);
}

int main(int argc, char *argv[])
{
    if (argc != 6) {
        fprintf(stderr, "Использование: %s <потоки> <строки> <столбцы> <окно> <K>\n",
argv[0]);
        return EXIT_FAILURE;
    }
    int threads = atoi(argv[1]);
    rows = atoi(argv[2]);
    cols = atoi(argv[3]);
    w = atoi(argv[4]);
    K = atoi(argv[5]);

    if (threads <= 0 || rows <= 0 || cols <= 0 || w <= 0 || K <= 0)
        die("все параметры должны быть положительными числами");
    if (w % 2 == 0)
        die("размер окна должен быть нечётным");

    size_t bytes = (size_t)rows * cols * sizeof(double);
    double *input = malloc(bytes);
    double *seq_a = malloc(bytes), *seq_b = malloc(bytes);
    double *par_a = malloc(bytes), *par_b = malloc(bytes);
```

```
    kernel = malloc(w * w * sizeof(double));
    if (!input || !seq_a || !seq_b || !par_a || !par_b || !kernel)
        die("не хватает памяти под матрицы");

    srand(42);
    for (int i = 0; i < rows * cols; i++)
        input[i] = rand() % 10000 / 100.0;
    for (int i = 0; i < w * w; i++)
        kernel[i] = 1.0 / (w * w);

    memcpy(seq_a, input, bytes);
    memcpy(par_a, input, bytes);

    printf("PID: %d (потоки: ps -T -p %d)\n", getpid(), getpid());

    double t0 = now_ms();
    run_sequential(seq_a, seq_b);
    double t_seq = now_ms() - t0;

    t0 = now_ms();
    run_parallel(par_a, par_b, threads);
    double t_par = now_ms() - t0;

    double *seq_res = (K % 2 == 0) ? seq_a : seq_b;
    double *par_res = (K % 2 == 0) ? par_a : par_b;
    int same = memcmp(seq_res, par_res, bytes) == 0;

    if (rows <= 20 && cols <= 20)
        save_steps("steps.txt", input);

    double S = t_seq / t_par;
    double X = S / threads;

    printf("Последовательно:   %.3f мс\n", t_seq);
    printf("Параллельно (%d):  %.3f мс\n", threads, t_par);
    printf("Ускорение S:       %.3f\n", S);
    printf("Эффективность X:   %.4f\n", X);
    printf("Результаты %s\n", same ? "совпадают" : "НЕ СОВПАДАЮТ!");

    free(input); free(kernel);
    free(seq_a); free(seq_b);
    free(par_a); free(par_b);
    return same ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Makefile

```
CC = gcc
CFLAGS = -O2 -Wall -pthread

THREADS = 1 2 4 6 8 12 16 128 1024
SIZE = 2048
WINDOW = 5
K = 10
RUNS = 3

main: main.c
	$(CC) $(CFLAGS) main.c -o main

test: main
	./main 2 5 5 3 3
	cat steps.txt

bench: main
	@echo "потоки прогон T_seq T_par результат" | tee results_$(SIZE).txt
	@for t in $(THREADS); do \
		for r in $$(seq $(RUNS)); do \
			./main $$t $(SIZE) $(SIZE) $(WINDOW) $(K) | awk -v t=$$t -v r=$
$r \
```

```
                          '/Последовательно/{ts=$$2} /Параллельно/{tp=$$(NF-
1)} /Результаты/{ok=$$2} END{print t, r, ts, tp, ok}' \
                          | tee -a results_$(SIZE).txt; \
            done; \
      done

clean:
      rm -f main steps.txt results_*.txt

.PHONY: test bench clean
```

Протокол работы программы

Тестирование:

```
$ make
gcc -O2 -Wall -pthread main.c -o main

$ ./main 2 5 5 3 3
PID: 17632 (потоки: ps -T -p 17632)
Шаги свёртки записаны в файл steps.txt
Последовательно:   0.002 мс
Параллельно (2):  0.288 мс
Ускорение S:       0.005
Эффективность X:   0.0027
Результаты совпадают

$ cat steps.txt
Матрица 5x5, окно 3x3, K = 3

Исходная матрица:
   61.66    27.40    88.81    32.41    10.12
   67.58    90.21    49.40     5.35    37.43
   58.74    31.43    97.17    45.04    36.96
   97.62    98.92    94.48     6.98    12.59
   38.67    63.52    78.09    10.33    57.27

После итерации 1:
   58.53    62.55    49.13    39.43    20.61
   58.33    63.60    51.91    44.74    27.98
   74.27    76.17    57.66    42.82    25.70
   64.88    73.18    58.44    48.77    30.67
   63.99    72.40    56.03    45.05    31.32

После итерации 2:
   59.94    57.14    51.39    38.11    29.11
   64.96    61.35    54.23    40.00    30.62
   67.55    64.27    57.48    43.19    33.89
   69.78    66.34    58.95    44.05    34.67
   67.08    64.59    58.59    44.74    36.16

После итерации 3:
   60.59    57.50    49.87    40.23    32.66
   63.07    59.81    51.91    42.00    34.28
   66.28    62.77    54.43    44.12    36.18
   67.11    63.85    55.80    45.75    37.94
   67.04    63.95    56.13    46.30    38.61
```

Ручная проверка первой итерации. Элемент [1][1] = 63,60: окно 3×3 целиком внутри матрицы, (61,66 + 27,40 + 88,81 + 67,58 + 90,21 + 49,40 + 58,74 + 31,43 + 97,17) / 9 = 572,40 / 9 = 63,60. Угловой элемент [0][0] = 58,53: за краем повторяется крайний элемент, (4 · 61,66 + 2 · 27,40 + 2 · 67,58 + 90,21) / 9 = 526,81 / 9 = 58,53. Оба значения совпадают с выводом программы.

```
$ ./main 12 2048 2048 5 10
PID: 5240 (потоки: ps -T -p 5240)
Последовательно:   663.500 мс
Параллельно (12):  142.669 мс
Ускорение S:       4.651
Эффективность X:   0.3876
Результаты совпадают

$ ./main 2 x 10 3 1
Ошибка: все параметры должны быть положительными числами
$ ./main 2 10 10 4 1
Ошибка: размер окна должен быть нечётным
```

Исследование ускорения и эффективности

Условия эксперимента

Замеры выполнены на ноутбуке с процессором Intel Core i5-13420H под Ubuntu в WSL 2. Процессор гибридный: 4 производительных ядра (P-ядра) с Hyper-Threading и 4 энергоэффективных ядра (E-ядра) без него, всего 8 физических и 12 логических ядер, L3-кэш 12 МБ.

Исследованы два набора входных данных с окном 5×5: матрица 2048×2048 при K = 10 (одна матрица занимает 32 МБ) и матрица 512×512 при K = 100 (2 МБ, помещается в кэш). Каждое число потоков запускалось 3 раза, в таблицах приведена медиана; Ts — медиана всех запусков последовательной версии в наборе. Во всех 54 запусках результаты параллельной и последовательной версий совпали.

Метрики

Ускорение показывает, во сколько раз параллельная версия быстрее последовательной:

$$S = T_s / T_p$$

где Ts — время работы последовательной версии, Tp — время работы параллельной версии на p потоках. В идеальном случае S = p.

Эффективность показывает, какую долю идеального ускорения даёт каждый поток:

$$X = S / p$$

где S — ускорение, p — число потоков. В идеальном случае X = 1, на практике X < 1.

Закон Амдала задаёт верхнюю оценку ускорения: $S \le 1 / (a + (1 - a) / p)$, где a — доля последовательной части программы (в нашем случае это создание потоков и синхронизация на барьере).

Результаты

Таблица 1. Матрица 2048×2048, окно 5×5, K = 10

Число потоков	Время исполнения, мс	Ускорение S	Эффективность X
последовательная	658,8	1,00	—
1	660,2	1,00	0,998
2	349,3	1,89	0,943
4	191,7	3,44	0,859
6	183,2	3,60	0,599
8	153,5	4,29	0,537
12 (= логических ядер)	**140,7**	**4,68**	**0,390**
16	159,5	4,13	0,258
128	141,2	4,66	0,036
1024	158,4	4,16	0,004

Таблица 2. Матрица 512×512, окно 5×5, K = 100

Число потоков	Время исполнения, мс	Ускорение S	Эффективность X
последовательная	410,7	1,00	—
1	411,8	1,00	0,997
2	221,3	1,86	0,928
4	136,4	3,01	0,753
6	127,8	3,21	0,535
8	110,9	3,70	0,463
12 (= логических ядер)	**95,8**	**4,29**	**0,357**
16	124,8	3,29	0,206
128	124,6	3,30	0,026
1024	672,5	0,61	0,0006

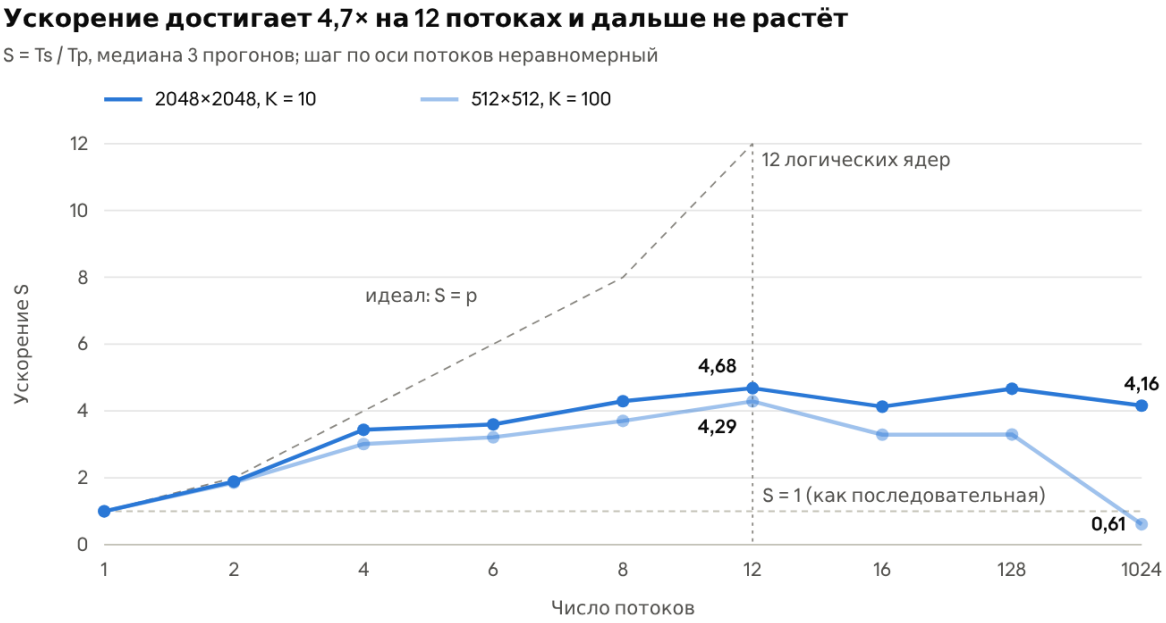


Рисунок 1. Зависимость ускорения от числа потоков

Анализ результатов

1. **До 4 потоков рост почти линейный** (S = 1,89 и 3,44, X = 0,94 и 0,86). Потоки, скорее всего, работают на 4 производительных ядрах, а строки матрицы считаются независимо, без синхронизации внутри итерации.
2. **От 4 до 12 потоков рост замедляется:** на 12 потоках S = 4,68 вместо идеальных 12. P-ядер всего 4; остальные потоки получают второй гиперпоток того же ядра или более медленное E-ядро. Строки делятся поровну, поэтому итерация длится столько, сколько работает самый медленный поток, а остальные ждут его на барьере. Кроме того, при нагрузке на все ядра ноутбучный процессор обычно снижает частоту.
3. **12 потоков (= числу логических ядер) дают максимум ускорения** в обоих наборах.
4. **16 потоков работают хуже, чем 12** (4,13 против 4,68; для 512×512 — 3,29 против 4,29). Часть ядер выполняет по два потока по очереди, и вся итерация ждёт эти потоки на барьере — это псевдопараллелизм.
5. **128 потоков на матрице 2048×2048 возвращают уровень 12 потоков** (S = 4,66). Каждому потоку достаётся 16 строк, и планировщик ОС отдаёт освободившимся быстрым ядрам следующие потоки, выравнивая нагрузку. Эффективность при этом всего 0,036.
6. **На 1024 потоках накладные расходы перевешивают выигрыш.** Для 2048×2048 при K = 10 ускорение падает до 4,16. Для 512×512 при K = 100

параллельная версия в 1,6 раза медленнее последовательной: половина потоков получает по одной строке, половина — ни одной, но все 1024 потока 100 раз синхронизируются на барьере.

На маленькой матрице ускорение ниже (4,29 против 4,68 на 12 потоках), хотя она помещается в кэш. Одна её итерация занимает около 4 мс, поэтому создание потоков и барьеры весомее на фоне полезной работы. Это согласуется с законом Амдала: чем больше доля последовательной части, тем ниже предел ускорения.

Вывод

В ходе лабораторной работы я реализовала последовательную и параллельную версии наложения фильтра на матрицу с помощью POSIX Threads и барьерной синхронизации между итерациями. Разделение матрицы по строкам позволило обойтись без мьютексов: каждый поток пишет только в свою часть результата. Максимальное ускорение составило 4,68 на 12 потоках; при числе потоков больше числа логических ядер ускорение не растёт, а эффективность падает почти до нуля. На результаты заметно влияет гибридная архитектура процессора с производительными и энергоэффективными ядрами.

При выполнении работы пришлось разобраться, как правильно измерять время многопоточной программы (clock() суммирует процессорное время всех потоков, поэтому использовалась clock_gettime) и как синхронизировать потоки между итерациями, не пересоздавая их на каждом шаге.